

Data Lab 实验报告

姓名: 舒楷迪

学号: 25300120163

一、实验截图

1. ./check_ops.py bits.c 运行结果

```
● asus@DESKTOP-0TF26NC:/mnt/c/Users/ASUS/Lab1-Datalab$ ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 8/8 operators
negativePart: 4/6 operators
copyByteWithin: 9/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 11/18 operators
secondLowestZeroBit: 5/8 operators
oddParity: 12/56 operators
rotateRightBits: 2/16 operators
roundEvenPow2: 17/24 operators
midpointTowardFirst: 22/32 operators
isBetweenEitherOrder: 23/48 operators
mul5Sat: 24/30 operators
classifyAdd3: 48/52 operators
floatScaleThreeHalves: 54/60 operators
floatRoundEven: 19/65 operators
float_i2f: 26/40 operators
bitCount: 28/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
```

2. ./btest 运行结果

```
● asus@DESKTOP-0TF26NC:/mnt/c/Users/ASUS/Lab1-Datalab$ ./btest
Score  Rating  Errors  Function
  1      1      0    signMask
  2      2      0    bitXor
  3      3      0   negativePart
  4      4      0   copyByteWithin
  4      4      0   logicalShift
  4      4      0   swapNibblePairs
  4      4      0 secondLowestZeroBit
  5      5      0    oddParity
  5      5      0 rotateRightBits
  5      5      0 roundEvenPow2
  5      5      0 midpointTowardFirst
  7      7      0 isBetweenEitherOrder
  7      7      0    mul5Sat
  7      7      0   classifyAdd3
  7      7      0 floatScaleThreeHalves
 10     10      0   floatRoundEven
 10     10      0   float_i2f
 10     10      0    bitCount
 10     10      0   bitReverse
Total points: 110/110
```

二、各函数实现思路

P1 signMask

返回最高位为 1 的掩码，即 0x80000000

直接使用 $1 \ll 31$ 即可，操作数 1 个

P2 bitXor

仅用 $\sim$ 和 $\&$ 实现异或

利用德摩根定律： $x \oplus y = \sim(\sim(x \& y) \& \sim(y \& x))$

先分别取出 x 为 0 且 y 为 1、 x 为 1 且 y 为 0 的位，再取反合并

P3 negativePart

若 $x < 0$ 返回 $-x$ ，否则返回 0

$x \gg 31$ 得到符号掩码（负数为全 1，非负为 0）

$\sim x + 1$ 是 $-x$ ，与符号掩码按位与即可

P4 copyByteWithin

将 x 的第 src 个字节复制到第 dst 个字节

计算源字节移位量 $s = src \ll 3$ ，目标移位量 $d = dst \ll 3$

取出源字节： $(x \gg s) \& 0xFF$

清除目标字节： $x \& \sim(0xFF \ll d)$

将源字节移到目标位置并或回去： $| (b \ll d)$

P5 logicalShift

实现逻辑右移。C 语言中 $\gg$ 是算术右移，负数高位补 1

先构造掩码 $\sim(((1 \ll 31) \gg n) \ll 1)$ ，它能保留低 $32-n$ 位为 1，高 n 位为 0

然后 $(x \gg n) \& mask$ 即可清除算术右移补的 1

P6 swapNibblePairs

交换每个字节内的高 4 位和低 4 位

构造掩码 $mask = 0x0F0F0F0F$ （通过 $0x0F$ 移位和或运算得到，避免大常量）

低 4 位左移 4 位，高 4 位右移 4 位，再或起来

P7 secondLowestZeroBit

找到第二低的 0 位并返回其掩码

$x | (x + 1)$ 会把最低的 0 位变成 1

然后 $\sim x1 \& (x1 + 1)$ 得到 $x1$ 中最低的 0 位，即原数的第二低 0 位

P8 oddParity

返回奇偶校验位：偶数个 1 返回 1，奇数个 1 返回 0

使用折叠异或：依次 $x \oplus x \gg 16$; $x \oplus x \gg 8$; ...，最终最低位是所有位的异或和

返回 $!(x \& 1)$ 即可

P9 rotateRightBits

循环右移 n 位

$n = n \& 31$ 保证移位量在 $0 \sim 31$

左移部分: $x \ll ((\sim n + 1) \& 31)$, 即左移 $(32-n) \bmod 32$

右移部分: 先算术右移 n 位, 再用掩码 $\sim(((1 \ll 31) \gg n) \ll 1)$ 清除高位补的 1, 得到逻辑右移结果

两部分或起来

P10 roundEvenPow2

将非负 x 舍入到最近的 2^n 倍数, 中间情况舍入到偶数倍

$M = 1 \ll n$, $half = M \gg 1$

$r = x \& (M + \sim 0)$ 取余数, $q = x \gg n$ 取商

判断是否恰好中间且商为偶数: $tie_down = !(q \& 1) \& !(r \wedge half)$

若 tie_down 为 1, 则 $x_adj = x - 1$, 否则 $x_adj = x$

最终 $((x_adj + half) \gg n) \ll n$

P11 midpointTowardFirst

计算 $(x+y)/2$ 的精确中点, 不溢出, 中间时靠近 x

$avg = (x \& y) + ((x \wedge y) \gg 1)$ 是无溢出的向下取整平均

$rem = (x \wedge y) \& 1$ 判断中点是否在两整数之间

安全判断 $x > y$: 同号时用 $(x - y) \gg 31$, 异号时看符号

$cond = rem \& gt$, 若 $x > y$ 且中点在中间, 则 $avg + 1$, 否则 avg

P12 isBetweenEitherOrder

判断 x 是否在 a 和 b 构成的闭区间内 (顺序任意)

安全计算 $x < a$ 和 $x < b$, 使用公式 $lt = ((diff \wedge temp) \gg 31) \& 1$

其中 $diff = u - v$, $temp = (u \wedge v) \& (diff \wedge u)$

同时判断 $x == a$ 或 $x == b$

若 $x < a$ 与 $x < b$ 真假不同, 或等于任一端点, 则返回 1

P13 mul5Sat

计算 $x * 5$, 溢出时饱和到 INT_MAX 或 INT_MIN

$x4 = x \ll 2$, 检查左移溢出: $!!((x4 \gg 2) \wedge x)$

$x5 = x4 + x$, 检查加法溢出: $((x \wedge x5) \gg 31) \& 1$

总溢出标志 $ovf = shift_ovf \mid add_ovf$

根据 x 符号选择饱和值 pos_sat 或 neg_sat

用掩码 $mask = (ovf \ll 31) \gg 31$ 选择饱和值或正常结果

P14 classifyAdd3

分类三个整数之和是否超出 int 范围

将 x, y, z 看作 64 位补码相加:

先算 $s1 = x + y$, 提取进位 $carry1$

再算 $s2 = s1 + z$, 提取进位 $carry2$

高 32 位 $H = (x \gg 31) + (y \gg 31) + (z \gg 31) + carry1 + carry2$, 低 32 位 $L = s2$

根据 H 和 L 的符号判断正溢出、负溢出或正常
返回 1、-1 或 0

P15 floatScaleThreeHalves

计算浮点数 $f * 3/2$ ，返回位级表示

分解符号、指数、尾数

处理 NaN、Inf、零

提取有效数字 M，乘以 3 得到 P

根据原数是否为次正规数分别处理：

次正规数结果可能仍为次正规数或变为正规数，需规格化并舍入到最近偶数

正规数直接计算 k，右移 P 并舍入，更新指数

若指数达到 255，返回无穷大 0x7F800000

P16 floatRoundEven

将浮点数舍入到最接近的整数，中间舍入到偶数

分解符号、指数、尾数

NaN、Inf 原样返回；次正规数或 $\text{exp} < 126$ 舍入为 0（保留符号）； $\text{exp} \geq 150$ 已是整数

计算 $\text{shift} = 150 - \text{exp}$ ，取整数部分 l 和余数 rem

根据 rem 与 half 的关系舍入，中间时若 l 为奇数则加 1

若 $l == 0$ 返回带符号的零

否则将整数 l 规格化为浮点数，返回结果

P17 float_i2f

将 int 转换为单精度浮点數位级表示

处理 0 和负数，取绝对值

找到最高有效位位置 exp

若 $\text{exp} \leq 23$ ，尾数左移对齐；否则右移并舍入到最近偶数

若尾数溢出到 $1 \ll 24$ ，则调整指数

组合符号、指数段和尾数返回

P18 bitCount

统计 x 中 1 的个数

使用 SWAR 分治法：

构造掩码 0x55555555、0x33333333、0x0F0F0F0F、0x00FF00FF、0x0000FFFF

依次合并相邻 1 位、2 位、4 位、8 位、16 位的计数

最终结果即为 1 的总数

P19 bitReverse

反转 32 位整数的所有位

分治法交换相邻 1、2、4、8、16 位

构造掩码： $m8 = 0x00FF00FF$ ， $m4 = m8 \gg (m8 \ll 4)$ ， $m2 = m4 \gg (m4 \ll 2)$ ， $m1 = m2 \gg (m2 \ll 1)$

依次交换： $x = ((x \gg k) \& m_k) \mid ((x \& m_k) \ll k)$

最后交换高低 16 位，用 mask16 模拟逻辑右移，避免算术右移补 1